

A Radix-4 Booth Multiplier in Structural VHDL

Architecture, optimization and synthesis at 32 bits in Nangate 45 nm

Erfan Taheri

Nangate 45 nm Open Cell Library · Synopsys Design Compiler · ModelSim

Abstract

This report presents the design and optimization of a signed radix-4 Booth multiplier described in structural VHDL, together with the synthesis results obtained for the 32×32 -bit instance using Synopsys Design Compiler and the Nangate 45 nm library. Three reduction strategies are implemented behind a single entity interface: a behavioural summation of the partial products, a Wallace carry-save adder tree, and a Dadda tree. Three architectural optimizations are then applied in sequence: elimination of the partial-product sign extension in favour of a constant correction vector, replacement of the Wallace tree by a generic Dadda schedule computed at elaboration time, and fusion of the Booth selection and negation logic into one Boolean function per partial-product bit. Each optimization is derived analytically, implemented as an alternative VHDL architecture, verified in simulation and measured over a sweep of clock constraints. A multiplier inferred by the synthesis tool from a plain $A * B$ statement is synthesised under identical conditions and used as a technology-mapped reference.

Across the sweep, the final architecture reaches **1.75 ns** [R4], while the inferred reference reaches **1.63 ns** [R6]. Their smallest netlists are **5029 μm^2** [R4] and **4441 μm^2** [R6], respectively. Relative to the unoptimized Wallace baseline, the three optimizations recover **69%** of the relaxed-area difference to the reference. At a common 3.0 ns synthesis target, post-synthesis activity analysis gives the final architecture 4.61 mW of dynamic power, 44.1% below the separate-negation Dadda variant [R8]. Tags such as [R4] identify the configuration report set; Appendix B gives the key.

Contents

1	Introduction	3
2	Architecture	3
2.1	Radix-4 modified Booth recoding	4
2.2	Datapath organisation	4
2.3	Exactness modulo 2^{2N}	4
3	Optimization 1: elimination of the partial-product sign extension	5
3.1	Row format and the cost of explicit sign extension	5
3.2	The inversion identity	5
3.3	Closed form of the correction constant	6
3.4	Absorption into the Booth increment row	6
3.5	Implementation	7
3.6	Measured effect	7
4	Optimization 2: a generic Dadda reduction schedule	7
4.1	Column representation	7
4.2	Wallace and Dadda placement criteria	8
4.3	Generation of the initial array	8

4.4	The scheduling algorithm	9
4.5	Carry accounting	9
4.6	Slot allocation	10
4.7	Consistency and generic cost	11
4.8	Constraints imposed by VHDL-93	11
4.9	Measured effect	12
5	Optimization 3: fusion of the Booth selector	12
5.1	Cost of the separate negation stage	12
5.2	Algebraic formulation	12
5.3	Completeness of the cover	13
5.4	Effect on technology mapping	13
5.5	Measured effect	13
6	Functional verification	13
7	Synthesis methodology	14
7.1	Flow and constraints	14
7.2	Derived zero-slack period	14
7.3	Post-synthesis activity and power	15
8	Results	15
8.1	Area and delay	15
8.2	Dynamic power and latency	16
8.3	Cell-level analysis	17
9	Conclusions	18
A	Project structure and variant selection	19
B	Measurement provenance	19

1 Introduction

The objective is to quantify how three structural choices change a hand-built signed multiplier under one controlled synthesis flow. A multiplier inferred from a plain $A * B$ statement provides a practical reference generated by the same tool and library, making the remaining area and timing differences measurable within this experiment.

The design is a radix-4 Booth multiplier written in structural VHDL, generic in the operand width N and instantiated at $N = 32$, signed, with the product taken modulo 2^{2N} . Three architectural optimizations are applied in sequence: elimination of the partial-product sign extension, a Dadda reduction schedule in place of a Wallace tree, and fusion of the Booth selection and negation steps. Each is derived analytically, implemented as an alternative VHDL architecture, verified against a reference model, and measured over a sweep of clock constraints.

Three constraints govern the implementation throughout:

- **Single source tree.** Every variant is an alternative **architecture** of an existing entity, selected by a VHDL **configuration**. No file is duplicated, so a variant differs from its predecessor only in the component it binds, and the measured difference is attributable to that component alone.
- **Controlled reference.** The inferred $A * B$ design is synthesised with the same script, constraints and library, so results are compared under the same conditions.
- **Verification precedes measurement.** A variant is synthesised only after it passes the self-checking testbench at both $N = 8$ and $N = 32$. Exhaustive testing at $N = 8$ covers every input pair; corner cases and 20 000 random pairs exercise the actual 32-bit elaboration.

Section 2 describes the architecture and establishes the modular-arithmetic framework on which the first optimization depends. Sections 3–5 derive, implement and measure the three optimizations. Section 6 describes the verification strategy, Section 7 the synthesis methodology, and Section 8 the measured results. Source organisation and the configuration mechanism are documented in Appendix A.

2 Architecture

The datapath is shown in Figure 1: Booth encoders drive one row generator each, a corrector supplies one additional row, a reduction tree compresses the $N/2 + 1$ rows to two, and a carry-propagate adder produces the $2N$ -bit product.

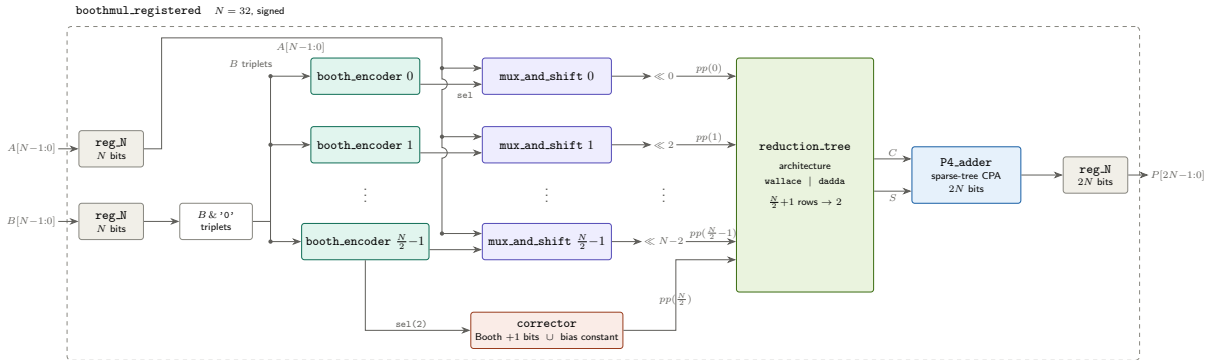


Figure 1: Top level of the structural multiplier. The partial-product rows are drawn generically: there are $N/2$ of them plus the corrector row. Only **reduction_tree** differs between the Wallace and Dadda variants, and only **mux_and_shift** differs between the sign-extension variants.

2.1 Radix-4 modified Booth recoding

A direct implementation of an $N \times N$ product accumulates one shifted copy of A per bit of B , giving N partial-product rows. Radix-4 modified Booth recoding inspects overlapping triplets of B , namely bits $(2i+1, 2i, 2i-1)$ with the convention $B_{-1} = 0$, and maps each triplet to a single signed digit $d_i \in \{0, \pm 1, \pm 2\}$, so that

$$A \cdot B = \sum_{i=0}^{N/2-1} d_i A \cdot 4^i. \quad (1)$$

triplet	digit d_i	sel	row generated
000, 111	0	000	all zeros
001, 010	+1	001	$+A$
011	+2	011	$+2A$
100	-2	111	$-2A$
101, 110	-1	101	$-A$

Table 1: Radix-4 modified Booth recoding as implemented in `booth_encoder`. The control vector is `sel = {neg, double, enable}`.

The number of partial-product rows is therefore $N/2$ rather than N . This is the principal benefit of the recoding because the cost and depth of the reduction network grow with the number of rows. Multiplication by two requires only a change in bit position, although the resulting wiring still contributes to routing after layout. Conditional negation adds logic and motivates two of the three optimizations presented below.

2.2 Datapath organisation

The datapath consists of five stages.

Stage	Entity	Function
encode	<code>booth_encoder</code>	triplet of $B \rightarrow \text{sel} = \{\text{neg}, \text{double}, \text{enable}\}$
generate	<code>mux_and_shift</code>	one partial-product row: $0, \pm A, \pm 2A$
correct	<code>corrector</code>	one additional row: Booth increments and the bias constant
reduce	<code>reduction_tree</code>	$N/2+1$ rows $\rightarrow$ carry and sum vectors
add	<code>P4_adder</code>	sparse-tree carry-propagate adder $\rightarrow 2N$ -bit product

Table 2: Stages of the structural multiplier. The synthesis top level, `boothmul_registered`, wraps this combinational block between input and output registers.

Row i carries weight 4^i , applied as a left shift of $2i$ positions in `boothmul`. The reduction tree is deliberately separated from the final carry-propagate adder, so that the reduction strategy can be replaced without touching any other file.

2.3 Exactness modulo 2^{2N}

Three mechanisms in this design discard information: the carry-save adders drop the carry generated by the most significant column, the final adder’s carry-out is left unconnected, and the sign-extension optimization of Section 3 deliberately introduces a bias that is cancelled later. All three are exact, for the same reason.

The datapath computes in the ring $\mathbb{Z}/2^{2N}\mathbb{Z}$. Every operation performed on the $2N$ -bit vectors (addition, carry-save addition, shifting, and the final carry-propagate addition) is a ring operation, so discarding any carry out of bit $2N-1$ is a reduction modulo 2^{2N} and commutes with everything

that follows. The product of two N -bit two's complement operands lies in $[-2^{2N-2}, 2^{2N-2}]$, which is representable in $2N$ bits, so its residue class modulo 2^{2N} determines it uniquely.

It is therefore sufficient to show that the array of rows delivered to the reduction tree is congruent to $A \cdot B$ modulo 2^{2N} . Intermediate values may overflow freely; only the congruence matters. This property is what makes the derivation in Section 3 valid, and it is the reason a constant may be added to the array to cancel a bias without any range analysis.

3 Optimization 1: elimination of the partial-product sign extension

3.1 Row format and the cost of explicit sign extension

The term $m_i = d_i A$ generated for row i lies in $[-2^N, 2^N - 2]$ and is therefore representable in $N + 1$ bits of two's complement. It must be placed in a $2N$ -bit array at weight 2^{2i} . Because m_i is signed and the array is summed as an unsigned quantity, bit N of the row must be replicated upward from position $N + 2i$ to position $2N - 1$.

The number of replicated bits in row i is $N - 1 - 2i$, so across the array

$$\sum_{i=0}^{N/2-1} (N - 1 - 2i) = \frac{N}{2}(N - 1) - \frac{N}{2}\left(\frac{N}{2} - 1\right) = \frac{N^2}{4} \quad (2)$$

partial-product bits exist only to carry sign information. At $N = 32$ this is 256 bits out of a total of 800, close to one third of the array. Each of them must be generated, routed, and compressed by the reduction tree, and none carries information beyond the single bit it replicates.

3.2 The inversion identity

Let s denote the sign bit of m_i , that is $m_{i,N}$. In two's complement,

$$m_i = -s \cdot 2^N + \sum_{j=0}^{N-1} m_{i,j} 2^j. \quad (3)$$

Since $s + \bar{s} = 1$ for a single bit, $-s = \bar{s} - 1$, and therefore

$$-s \cdot 2^N = \bar{s} \cdot 2^N - 2^N. \quad (4)$$

Define the *stored* row as

$$R_i = \bar{s} \cdot 2^N + \sum_{j=0}^{N-1} m_{i,j} 2^j, \quad (5)$$

that is: the N low bits unchanged, the sign bit inverted, and zeros above. Substituting (4) gives the exact relation

$$m_i = R_i - 2^N. \quad (6)$$

The sign extension has been replaced by a single inverted bit. What remains is a constant error of exactly 2^N per row, independent of A , of the Booth digit, and of the row index. It is not an approximation, and it does not depend on the data.

3.3 Closed form of the correction constant

Applying (6) to (1), the array of stored rows overshoots the product by

$$E = \sum_{i=0}^{N/2-1} 2^{N+2i} = 2^N \cdot \frac{4^{N/2} - 1}{3} = \frac{2^N(2^N - 1)}{3}. \quad (7)$$

By Section 2.3 the correction is the additive inverse of E in $\mathbb{Z}/2^{2N}\mathbb{Z}$:

$$C = 2^{2N} - \frac{2^N(2^N - 1)}{3} = 2^N \left(2^N - \frac{2^N - 1}{3} \right). \quad (8)$$

Two properties of (8) are structurally important. First, C is a multiple of 2^N , so its lower N bits are zero. Second, for even N the quantity $(2^N - 1)/3$ has the binary form $0101 \dots 01$, and subtracting it from 2^N yields $1010 \dots 1011$. The constant therefore occupies bits $2N - 1$ down to N with the hexadecimal pattern $\text{AA} \dots \text{AB}$.

N	E	C , bits $2N-1$ down to N
4	80	0xB
8	21 760	0xAB
16	1.43×10^9	0xAAAAB
32	6.14×10^{18}	0xAAAAAAAB

Table 3: The correction constant for several operand widths. The repeating pattern is a direct consequence of the factor $1/3$ in (7).

The constant is generated at elaboration time by `sign_ext_const` in `common_pkg`. It is built bit by bit rather than arithmetically, because C is a $2N$ -bit quantity and the VHDL integer type cannot represent it at $N = 32$:

```
function sign_ext_const(NB : natural) return std_logic_vector is
  variable v : std_logic_vector(2*N-1 downto 0) := (others => '0');
begin
  v(NB) := '1';
  for j in NB+1 to 2*N-1 loop
    if (j mod 2) = 1 then v(j) := '1'; end if;
  end loop;
  return v;
end function;
```

3.4 Absorption into the Booth increment row

Radix-4 Booth already requires one additional row. A negated row is produced as a one's complement by `mux_and_shift`; the matching $+1$ is not added locally but collected across all rows, with `sel(2)` of row i placed at weight 2^{2i} . This increment row occupies only the even positions of the lower half of the array.

The correction constant C occupies only the upper half. The two therefore share one row without collision, and the entire cost of Optimization 1 is confined to the upper half of a row the design already had. The `corrector` entity builds it:

```
architecture no_sign_extend of corrector is
  constant SE_CONST : std_logic_vector(2*N-1 downto 0) := sign_ext_const(N);
begin
  gen_neg: for i in 0 to N/2-1 generate
    correction_v(2*i) <= neg_bits(i);
    correction_v(2*i+1) <= '0';
  end generate;
```

```
correction_v(2*N-1 downto N) <= SE_CONST(2*N-1 downto N); -- high half
end architecture;
```

The high half is a tie to the supply rails. It contains no logic, and the synthesis tool propagates it as constants through the first reduction stage.

3.5 Implementation

The two row formats are alternative architectures of `mux_and_shift`. The baseline sign-extends to $2N$ bits; the optimized version constructs the row in $N + 1$ bits, inverts bit N , and zero-fills above:

```
-- architecture sign_extend
q := std_logic_vector(resize(signed(A), 2*N)); -- full-width row
...
q := q xor (q'range => sel(2));
pp <= q;

-- architecture no_sign_extend
q := std_logic_vector(resize(signed(A), N+1)); -- N+1 bits only
...
q := q xor (q'range => sel(2));
q(N) := not q(N); -- stored sign
pp <= std_logic_vector(resize(unsigned(q), 2*N));
```

The two row formats define two corrector architectures. The `no_sign_extend` format biases every row by $+2^{2i+N}$, which `corrector(no_sign_extend)` cancels; the baseline format has no bias and takes `corrector(sign_extend)`. Each pair is bound as a unit by the configurations in `cfg/`.

3.6 Measured effect

The array shrinks from 800 to 561 partial-product bits, a reduction of 29.9%. The minimum area observed in the sweep falls from $6346 \mu\text{m}^2$ [R1] to $5975 \mu\text{m}^2$ [R2], a reduction of 5.8%, and the cell count at a 3.0 ns constraint falls from 4639 to 4455 [R7].

The area reduction is much smaller than the reduction in partial-product bits. The removed bits are constant-valued sign replications, and constant propagation in `compile_ultra` eliminates most of the associated logic independently of the optimization. The contribution that constant propagation cannot make is the reduction in structural column height, which determines the size of the reduction network; that is what Optimization 2 exploits.

4 Optimization 2: a generic Dadda reduction schedule

This section replaces the row-wise Wallace tree with a Dadda schedule computed at elaboration time from the geometry of the partial-product array. The package construction supports the even operand widths used by radix-4 recoding; the reported synthesis results use $N = 32$.

4.1 Column representation

The partial-product array is a set of $2N$ columns; column k contains n_k bits, referred to below as *dots*, all of weight 2^k . A reduction network compresses every column until no column contains more than two dots, at which point a carry-propagate adder completes the sum.

The two primitives are the full adder, which takes three dots of column k and produces one dot in column k (sum) and one in column $k + 1$ (carry), and the half adder, which takes two and produces the same two outputs. A full adder reduces the total dot count by one; a half adder does not reduce it at all.

This gives an exact invariant, used as a correctness check in Section 4.7: if the array begins with D_0 dots and terminates with D_f , the number of full adders required is exactly $D_0 - D_f$, independently of the schedule. Half adders fall outside this accounting: they occupy cells without contributing to the reduction, so the quality of a schedule is measured by how few of them it places.

4.2 Wallace and Dadda placement criteria

In the classical algorithms, Wallace reduction compresses as early as possible, whereas Dadda reduction delays compression until a column exceeds the target height for the next stage. The RTL baseline used here is a row-wise Wallace implementation: it instantiates full-width carry-save adders, including bit positions where one or more inputs are constant. The Dadda implementation is column-aware and instantiates only the compressors required by its schedule.

The targets follow the Dadda sequence

$$d_0 = 2, \quad d_{j+1} = \left\lfloor \frac{3}{2} d_j \right\rfloor \implies 2, 3, 4, 6, 9, 13, 19, 28, \dots \quad (9)$$

where d_j is the largest column height reducible to two in j stages. The number of stages needed for r rows is the smallest j with $d_j \geq r$. For $N = 32$ the array has $N/2 + 1 = 17$ rows, so $d_6 = 19 \geq 17$ and six reduction stages are required, with targets 13, 9, 6, 4, 3, 2.

Both quantities are implemented in `dadda_types_pkg`:

```
function height_dadda(lev : natural) return natural is
  variable temp : integer := 2;
begin
  for i in 0 to lev-1 loop temp := (temp*3)/2; end loop;
  return temp;
end function;

function num_layers_dadda(NROWS : natural) return natural is
  variable temp : natural := 2; variable level : natural := 1;
begin
  while NROWS > temp loop
    temp := height_dadda(level);
    level := level + 1;
  end loop;
  return level;
end function;
```

The package header declares `NUM_LAYERS_D` as a deferred constant. The package body sets its value after defining `num_layers_dadda`, so later declarations in the header can use the constant.

In the Dadda scheduler, a half adder is needed only when a column is exactly one dot above its target. For $N = 32$ the generated schedule uses 45 half adders. The Wallace baseline does not explicitly instantiate half adders; its full-width carry-save adders instead leave synthesis to simplify positions with constant inputs. Table 5 therefore compares instantiated RTL compressors rather than claiming a one-to-one post-mapping cell count.

4.3 Generation of the initial array

The schedule is derived from the geometry of the array, not from its values, so it can be computed before any signal exists. The function `making_initail_layer` returns a Boolean occupancy matrix for the sign-extension-eliminated layout of Section 3:

```
-- rows 0 .. N/2-1 : N+1 significant bits, shifted by 2i
loop_rows: for ro in 0 to (NBIT/2-1) loop
  for col in ro*2 to ro*2+NBIT loop
    rows(ro)(col) := '1';
  end loop;
```

```

end loop;

-- row N/2 : the corrector row
for i in 0 to ((NBIT-1)/2) loop
    rows(last_row)(i*2) := '1';
end loop;
rows(last_row)(NBIT) := '1';
for i in 0 to ((NBIT-1)/2) loop
    rows(last_row)(i*2 + NBIT+1) := '1';
end loop;

```

-- Booth increments, even weights
-- the trailing B of the constant
-- the repeating A above it

The three parts of the corrector row correspond term by term to the decomposition of Section 3: the Booth increments in the low half, the bit at position N , and the alternating pattern above it. The matrix encodes the optimized layout, so the Dadda architecture is defined for the sign-extension-eliminated row format only, and is bound with `mux_and_shift(no_sign_extend)` or `mux_and_shift(fused_selector)`.

4.4 The scheduling algorithm

`constructing_dadda` performs the entire schedule at elaboration time and returns three matrices of identical shape, indexed by (layer, row slot, column):

- **FA_matrix**: marks the three source slots of every full adder;
- **HA_matrix**: marks the two source slots of every half adder;
- **remaining_system**: the dots no compressor consumed, which pass unchanged to the next layer.

The three are produced in a single pass. A mark-and-clear discipline guarantees that they stay mutually consistent: when a dot is assigned to a compressor its position is written into the corresponding matrix and simultaneously cleared from the working system, so it can neither be assigned twice nor survive into the set of passed dots.

```

loop_setting_FAs: for i in 0 to (fa_id-1) loop
    node_cnt := 0; roww := 0;
    while node_cnt < 3 loop
        if system(lev)(roww)(col) = '1' then
            node_cnt := node_cnt + 1;
            FA_matrix(lev)(roww)(col) := '1'; -- record the source slot
            system(lev)(roww)(col) := '0'; -- and consume it
        end if;
        roww := roww + 1;
    end loop;
end loop;

```

Layers are processed from the initial layer downward, and columns within a layer from least to most significant, because the compressor count of a column depends on the carries produced by the column below it.

4.5 Carry accounting

The height test is the subtle part of the algorithm. The target height applies to layer $\text{lev} - 1$, and column k of that layer receives dots from three distinct sources:

1. C_{in} carries generated by column $k - 1$ of the *same* stage, written into layer $\text{lev} - 1$ when column $k - 1$ was processed;
2. one sum bit per compressor placed in column k ;
3. the dots of column k that no compressor consumed.

If column k holds n dots at layer lev and is given F full adders and H half adders, the resulting height at layer $\text{lev} - 1$ is

$$C_{\text{in}} + (F + H) + (n - 3F - 2H) = C_{\text{in}} + n - 2F - H, \quad (10)$$

so the compressor count must satisfy $C_{\text{in}} + n - 2F - H \leq d_{\text{lev}-1}$. A column is therefore sized against a quantity that does not exist in its own layer. A test that counts only the dots of column k satisfies the target locally but not globally: columns exceed their target height and the array does not reach two rows within the allotted number of stages.

The loop implements exactly this, using the fact that a full adder removes two dots from the height and a half adder removes one:

```
count_temp := count + C_out_count;      -- own dots plus carries from col-1
while (count_temp > max_height) loop
  if count_temp >= (max_height + 2) then
    count_temp := count_temp - 2; fa_id := fa_id + 1;
  elsif count_temp = (max_height + 1) then
    ha_flag := 1; count_temp := count_temp - 1;
  end if;
end loop;
```

The half-adder branch is reached only when the residual excess is exactly one, which is the property that keeps the half-adder count low.

4.6 Slot allocation

A compressor instantiated in the netlist must know which row slot of the next layer its outputs drive. The allocation order for column k of layer $\text{lev} - 1$ is fixed:

slots $0 \dots C_{\text{in}} - 1$	carries arriving from column $k - 1$
slots $C_{\text{in}} \dots C_{\text{in}} + F + H - 1$	sums of this column's compressors
remaining slots	dots passed through unchanged

This order must be reproduced identically in two places: in `constructing_dadda`, which writes the matrices, and in `dadda_tree`, which writes the port maps. The difficulty is that a `generate` statement carries no sequential state. Each instance is elaborated independently and cannot inherit a running counter from the previous column, so every offset must be recomputable from the matrices alone. This is the purpose of the analysis functions in `dadda_math_pkg`.

Function	Returns
<code>num_rows_dadda(lev)</code>	occupied row slots at layer lev
<code>fa_count_col_dadda(...)</code>	number of full adders in a column
<code>ha_flag_col_dadda(...)</code>	1 if the column has a half adder, else 0
<code>num_C_out_from_prev_col(...)</code>	C_{in} , recomputed from column $k - 1$
<code>fa_arg_func(...)</code>	the three source row indices of each full adder
<code>ha_arg_func(...)</code>	the two source row indices of the half adder
<code>remaining_counting_func(...)</code>	number of passed dots in a column
<code>remaining_bit_pos(...)</code>	their row indices

Table 4: Analysis functions used to recover port-map positions from the scheduling matrices. All are pure and evaluated at elaboration time.

`num_C_out_from_prev_col` resolves the generate-statement constraint directly: instead of propagating C_{in} forward, it recomputes it by counting the compressors of column $k - 1$ in the same layer. Two assertions enforce the schedule invariants at elaboration time: at most one half adder per column and unique half-adder source positions. A violation therefore terminates elaboration rather than producing a mis-wired tree.

The structural instantiation then reads directly from the matrices. The generate nest is three levels deep (layer, column, compressor), and its constant declarations carry all the positional information:

```
columns: for col in 0 to NBIT*2-1 generate
  constant fa_count_col    : integer := fa_count_col_dadda(FA_matrix, lev, num_rows_current, col);
  constant C_out_counting  : integer := num_C_out_from_prev_col(FA_matrix, HA_matrix, lev, col);
  constant FA_args         : fa_args_t(0 to fa_count_col-1)
                           := fa_arg_func(FA_matrix, num_rows_current, fa_count_col, lev, col);
begin
  mapping_fa: for ID in 0 to fa_count_col-1 generate
    constant sum_bit_arg   : integer := ID + C_out_counting;
    constant carry_out_arg : integer := ID;
  begin
    FA_i : FA port map (
      A => layer(lev)(FA_args(ID)(0))(col),
      B => layer(lev)(FA_args(ID)(1))(col),
      Ci => layer(lev)(FA_args(ID)(2))(col),
      S => layer(lev-1)(sum_bit_arg)(col),
      Co => layer(lev-1)(carry_out_arg)(col+1));
  end generate;
end generate;
```

4.7 Consistency and generic cost

The schedule was validated against the dot-count invariant. For the optimized layout at $N = 32$ the array begins with $D_0 = 561$ dots and terminates with $D_f = 126$, so exactly $561 - 126 = 435$ full adders are necessary; the schedule produces 435. The identity holds at every width tested, and row 2 of the final layer is empty at all widths, confirming that the reduction really does terminate at two rows.

N	rows	array dots		stages	Dadda			Wallace	
		baseline	optimized		FA	HA	cells	FA	instances
8	5	56	45	3	15	9	24		48
16	9	208	153	4	91	21	112		224
32	17	800	561	6	435	45	480		960
64	33	3136	2145	8	1891	93	1984		3968

Table 5: Cost of the reduction network as a function of operand width, computed from the elaboration-time schedule. The Wallace column gives the number of full adder instances written by the row-wise implementation: $2N$ per carry-save adder, with no allowance for columns that are not full.

Table 5 quantifies the structural difference. The Wallace implementation instantiates carry-save adders across the full $2N$ width, giving 960 full adder instances at $N = 32$ against 435 full adders and 45 half adders for Dadda. Many of the Wallace instances have constant inputs and are removed by the synthesis tool, so the difference in the synthesised result is smaller than the instance-count ratio suggests. The measured Dadda result is also faster, but the reports do not isolate how much of that improvement comes from compressor placement, logic depth or subsequent technology mapping.

4.8 Constraints imposed by VHDL-93

Three language restrictions shaped the implementation.

Generate bounds must be globally static. The number of compressors in a column is data of the schedule, so the schedule must be available as a constant expression before elaboration of

the generate statement. This is why it is computed by a pure function over Boolean matrices rather than derived incrementally while the structure is built.

A package cannot see a generic. The port type `pp_array` must be declared in a package so that both the entity and its instantiators can see it, and a VHDL-93 package has no visibility of generics. The operand width is therefore a constant, `NBIT` in `common_pkg`, rather than a generic of the top-level entity. The design remains generic in the sense that every downstream quantity is a function of `NBIT`; changing that one constant reconfigures the multiplier, the reduction schedule and the testbench together.

Deferred constants. `NUM_LAYERS_D` is required by the type declarations in the package header but computed by a function defined in the package body. It is declared without a value in the header and given one in the body.

4.9 Measured effect

The minimum area falls from $5975 \mu\text{m}^2$ [R2] to $5620 \mu\text{m}^2$ [R3], a further 5.9%, and the fastest derived zero-slack period improves from 1.87 ns to 1.73 ns [R3], the largest single improvement in delay among the three optimizations. The cell count at a 3.0 ns constraint falls from 4455 to 3936 [R7].

5 Optimization 3: fusion of the Booth selector

5.1 Cost of the separate negation stage

After Optimization 2 the design is $1180 \mu\text{m}^2$ larger than the inferred reference. A cell census of the two netlists at a 3.0 ns constraint [R7] localises the difference.

	XOR + XNOR	FA cells	INV	total cells
Structural, Dadda	1230	117	505	3936
Inferred $A * B$	559	268	311	3035

Table 6: Cell census at a 3.0 ns constraint [R7], before Optimization 3.

The structural design contains 671 more exclusive-or cells than the inferred one and only 117 `FA_X1` cells, despite the 435 full adders instantiated in the Dadda tree. The RTL row generator provides a plausible cause: a multiplexer selects A , $2A$ or zero, then $N + 1$ conditional inversions apply `sel(2)`. Across 16 rows, the source contains 528 such bit-level inversions before the reduction tree.

This boundary matters to technology mapping because full-adder functions are also XOR-rich. Mixing conditional inversion with compression gives the mapper a larger Boolean cone to restructure. The row generator is changed in isolation, and the resulting mapped cell families are compared in Table 7.

5.2 Algebraic formulation

The multiplexer and the conditional inversion can be merged into one function. Let

$$\text{src}_1 = A_{N-1} \parallel A \quad (A \text{ in } N + 1 \text{ bits}) \quad (11)$$

$$\text{src}_2 = A \parallel '0' \quad (2A \text{ in } N + 1 \text{ bits}) \quad (12)$$

and decode the control vector once per row into four mutually exclusive conditions:

$$\begin{aligned}
c_{+1} &= \text{sel}(0) \cdot \overline{\text{sel}(1)} \cdot \overline{\text{sel}(2)}, & c_{-1} &= \text{sel}(0) \cdot \overline{\text{sel}(1)} \cdot \text{sel}(2), \\
c_{+2} &= \text{sel}(0) \cdot \text{sel}(1) \cdot \overline{\text{sel}(2)}, & c_{-2} &= \text{sel}(0) \cdot \text{sel}(1) \cdot \text{sel}(2).
\end{aligned} \tag{13}$$

Each output bit is then a four-term sum of products:

$$q_i = c_{+1} \text{src}_{1,i} + c_{-1} \overline{\text{src}_{1,i}} + c_{+2} \text{src}_{2,i} + c_{-2} \overline{\text{src}_{2,i}}, \tag{14}$$

after which sign-extension elimination is applied exactly as before: $pp_N = \overline{q_N}$, $pp_{N-1:0} = q_{N-1:0}$, and zeros above.

5.3 Completeness of the cover

Equation (14) enumerates four of the five possible row values; the zero row is covered implicitly, because all four conditions are false when $\text{sel}(0) = 0$ and the sum of products evaluates to zero. The cover is complete provided the encoder never asserts $\text{sel}(2)$ with $\text{sel}(0)$ deasserted; that is, it never requests a negated zero row. Table 1 confirms this: the only codes with $\text{sel}(2) = 1$ are 101 and 111, both of which have $\text{sel}(0) = 1$. The property follows from the encoding table and is not an assumption imposed on it.

5.4 Effect on technology mapping

Equation (14) exposes selection and conditional inversion as one sum-of-products function per bit. The source no longer places an explicit XOR stage between the selector and the reduction tree, giving Design Compiler the opportunity to use the Nangate library’s compound AOI/OAI cells and to share complemented forms of A across rows.

	XOR + XNOR	FA cells	AOI/OAI	total cells	area
Dadda, separate negation	1230	117	1405	3936	5725 μm^2
Dadda, fused selector	85	448	1311	2819	5125 μm^2
Inferred $A * B$	559	268	950	3035	4910 μm^2

Table 7: Cell census at a 3.0 ns constraint [R7]. Counts cover the entire mapped design, including the final carry-propagate adder.

Exclusive-or cells fall from 1230 to 85, while FA_X1 cells rise from 117 to 448. Together, the two shifts show that removing the explicit XOR boundary allowed the mapper to retain more adder structure. This single-architecture rewrite produces the largest area reduction of the three optimizations.

5.5 Measured effect

Minimum area falls from 5620 μm^2 [R3] to 5029 μm^2 [R4], a reduction of 10.5%. The fastest derived period is 1.75 ns, compared with 1.73 ns for separate-negation Dadda [R3,R4]; the fusion therefore trades 0.02 ns at the fastest measured edge for a much smaller netlist. Total cell count at the 3.0 ns target falls by 28%.

6 Functional verification

All variants share one self-checking testbench, `tb_multiplier`, which reads the operand width from `common_pkg` and selects its coverage mode accordingly.

The reference model is `signed(A) * signed(B)` truncated to $2N$ bits, evaluated independently of the design. Mismatches are counted rather than asserted, so a failing variant reports the number of failing vectors.

NBIT	coverage
≤ 8	exhaustive: all 2^{2N} operand pairs (65 536 at $N = 8$)
> 8	corner cases $\{0, \pm 1, 2^{N-1} - 1, -2^{N-1}\}$ plus 20 000 random pairs

Every variant is verified exhaustively at $N = 8$ and then exercised at $N = 32$ with corner cases and 20 000 random operand pairs. The smaller run gives complete input coverage for one elaborated width; the larger run directly tests the width used for synthesis. Together they exercise the recoding table, bias cancellation, fused selector and generated column schedule.

A separate post-synthesis testbench drives 500 deterministic operand pairs through each mapped netlist with maximum-delay SDF. It retains a signed multiplication checker, but its main purpose is consistent switching-activity capture for the power comparison in Section 7.3.

Each synthesis configuration has a matching testbench configuration (Appendix A), so the binding verified in simulation is the binding that is synthesised.

7 Synthesis methodology

7.1 Flow and constraints

The synthesis top level is `boothmul_registered`, which places the combinational multiplier between input and output registers. This makes the critical path internal, from register to register through the multiplier, so that the reported timing measures the design rather than the test environment.

Library	NangateOpenCellLibrary, typical ECSM
Tool	Synopsys Design Compiler W-2024.09-SP2, <code>compile_ultra</code>
Wire load model	<code>5K_hvrat1o_1_4</code>
Clock	<code>create_clock -period \$clockPeriod, swept</code>
Clock uncertainty	0.05 ns; transition 0.05 ns; latency 0.05 ns
Input delay	0.25 ns, absolute
Output delay	0.15 ns, absolute
Driving cell	<code>BUF_X4</code> ; output load 0.05 pF
Max transition	0.20 ns
Area constraint	<code>set_max_area 0</code>
Synthesis sweep	21 common periods from 1.0 to 6.0 ns, 6 configurations
Additional points	0.4–0.9 ns for the two Dadda variants

Table 8: Synthesis environment. The same constraint file is sourced once per period; `$clockPeriod` is the only quantity that changes.

The input and output delays are absolute values rather than fractions of the clock period. This is deliberate and is a precondition for the metric defined in Section 7.2: if the port constraints scaled with the period, the slack reported at one period could not be compared with the slack reported at another. Because the design is registered on both sides, the port paths terminate at a flip-flop within a few gates and never become critical, even at the tightest constraint. The timing reports confirm this: the critical path runs from a bit of the B register to the most significant bit of the product register in every run of the sweep.

The clock and reset networks are marked `set_dont_touch_network` and `set_ideal_network`, since clock tree construction belongs to place-and-route and not to this flow.

7.2 Derived zero-slack period

For a fixed netlist, the timing report gives the clock period at which its critical setup path would have zero slack:

$$T_0 = T_{\text{constraint}} - \text{slack}. \quad (15)$$

Slack is negative for a violated path, so a violated run yields $T_0 > T_{\text{constraint}}$. The port delays in Table 8 are absolute, and clock uncertainty, transition and latency are fixed throughout the sweep.

As an example, the fused-selector variant constrained at 1.0 ns reports a slack of -0.757 ns [R4], giving $T_0 = 1.757$ ns for that netlist. The tighter-target extension produces a slightly smaller T_0 of 1.75 ns.

7.3 Post-synthesis activity and power

The power flow uses the mapped netlist, its SDC and its maximum-delay SDF for each synthesis point. The gate-level testbench applies the same 500-vector operand sequence to every variant and runs the clock at that netlist’s derived T_0 , rather than at the original requested period. A VCD captures internal activity; `vcd2saif` converts it to SAIF, and Power Compiler annotates the netlist before `report_power`. The committed power set contains 20 matched points per variant from 1.0 to 6.0 ns.

All 120 `report_saif` outputs report 100% user annotation for nets, ports and pins [R8]. Dynamic power is the sum of cell-internal and net switching power; leakage is reported separately. Because the clock rate changes with T_0 , power must be interpreted together with latency. The power plot is therefore a second Pareto front, computed directly from (T_0, P_{dyn}) rather than by reusing the area front.

8 Results

8.1 Area and delay

Variant	fastest T_0	area there	min. area	gap closed	source
Wallace, sign-extended (baseline)	1.86 ns	10022 μm^2	6346 μm^2	n/a	[R1]
+ sign-extension elimination	1.87 ns	9234 μm^2	5975 μm^2	19.5%	[R2]
+ Dadda reduction	1.73 ns	9086 μm^2	5620 μm^2	38.1%	[R3]
+ fused selector	1.75 ns	7988 μm^2	5029 μm^2	69.1%	[R4]
Behavioural Booth (adder chain)	2.00 ns	7986 μm^2	6210 μm^2	7.1%	[R5]
Inferred A * B	1.63 ns	5337 μm^2	4441 μm^2	reference	[R6]

Table 9: Summary of the sweep. T_0 is the derived zero-slack period from (15). *Gap closed* uses the minimum-area column.

Table 9 summarises the measured area and timing results. The three optimizations together recover 69.1% of the area difference between the structural baseline and the inferred reference. The fused design reduces the delay difference from 0.23 ns to 0.12 ns, while plain Dadda reaches the fastest structural point at 1.73 ns.

The behavioural Booth variant, which replaces the reduction tree with a chain of signed additions, has the slowest derived period in the sweep at 2.00 ns [R5]. It is nevertheless smaller than the Wallace baseline at relaxed timing, showing how extensively the synthesis tool can restructure an adder chain. Its timing result also shows that inference does not compensate fully for a serial high-level description.

Figure 2 shows the full fronts. Sign-extension elimination and Dadda reduction improve most of the measured range. The fused selector gives the smallest area, while separate-negation Dadda reaches the fastest structural point.

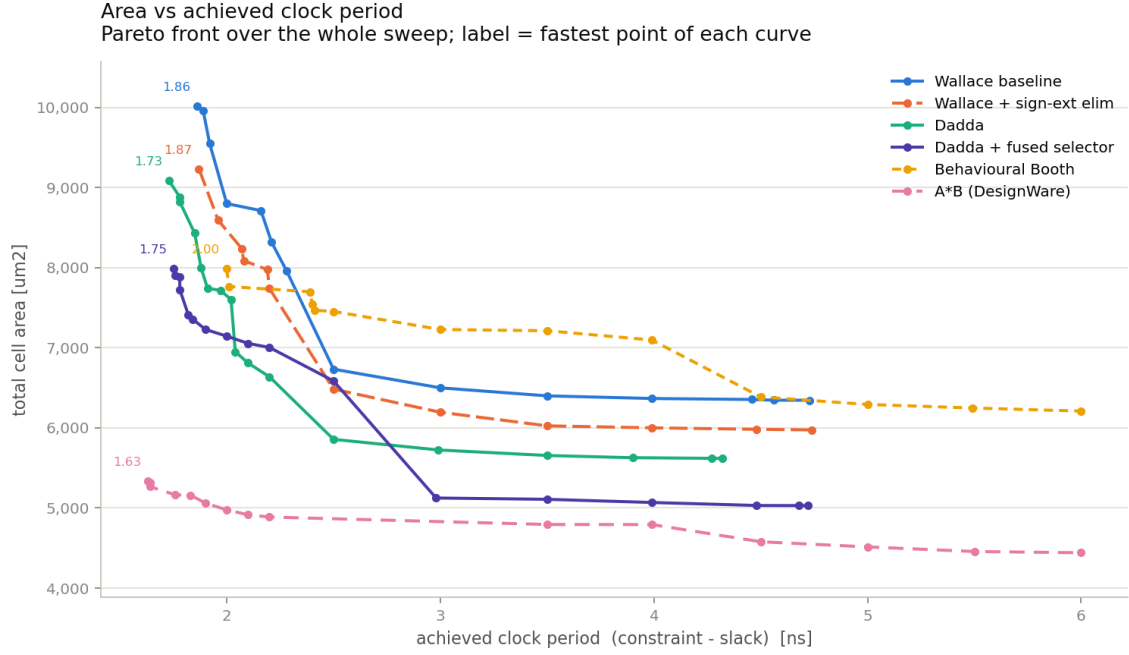


Figure 2: Area against derived zero-slack period. Each curve is the Pareto front of one variant over all available runs; dominated runs are omitted. All variants have 21 common targets, and the two Dadda variants have six additional tight targets. Lower and further left is better.

8.2 Dynamic power and latency

Variant at the 3.0 ns target	achieved period	dynamic	leakage
Wallace, sign-extended	3.000 ns	9.38 mW	0.148 mW
+ sign-extension elimination	3.000 ns	8.58 mW	0.138 mW
+ Dadda reduction	2.993 ns	8.25 mW	0.129 mW
+ fused selector	2.983 ns	4.61 mW	0.119 mW
Behavioural Booth	2.998 ns	5.87 mW	0.158 mW
Inferred A * B	2.998 ns	3.22 mW	0.103 mW

Table 10: Activity-based power at a common synthesis target [R8]. The small differences in achieved period reflect each mapped netlist’s timing slack.

At this point the fused selector reduces dynamic power by 44.1% relative to separate-negation Dadda and by 50.8% relative to the Wallace baseline. It is 43.3% above the inferred reference. The large change is consistent with the cell-count and XOR/XNOR reductions in Table 7.

Figure 3 shows the expected frequency dependence, but it also separates architectures at comparable latency. The fused-selector curve stays below the separate-negation Dadda curve throughout their measured overlap. The inferred multiplier remains the lowest-power implementation in the study.

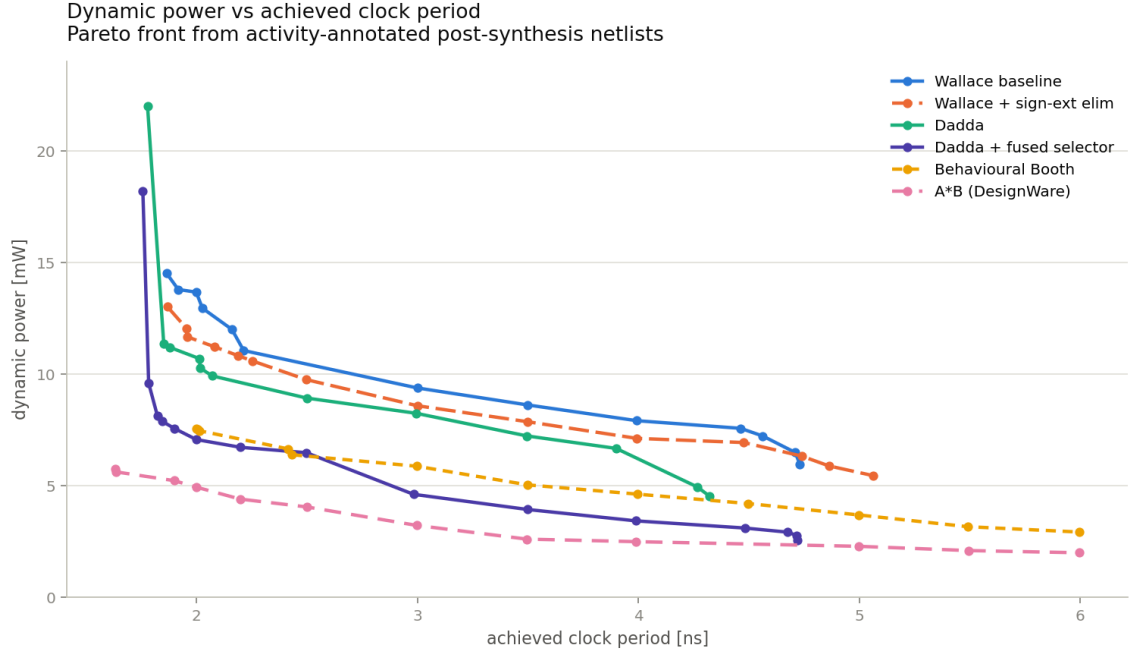


Figure 3: Dynamic power against achieved period for the activity-annotated post-synthesis netlists. Each line is that variant’s nondominated (T_0, P_{dyn}) subset; lower and further left is better.

8.3 Cell-level analysis

Table 7 gives the cell census at a fixed 3.0 ns constraint. Two properties of the sequence follow from it.

The area reduction from Optimization 1 (5.8%) is much smaller than its reduction in partial-product bits (29.9%), because constant propagation removes most of the logic associated with sign replication independently of the optimization. Its contribution is the reduction in structural column height, which is realised by Optimization 2.

The three optimizations are not independent. The gain of Optimization 3 depends on Optimization 2, because the compressors the mapper matches after the XOR layer is removed are the ones the Dadda schedule instantiates. Measured in isolation, each optimization is smaller than it is in sequence.

9 Conclusions

A signed 32×32 radix-4 Booth multiplier was implemented in structural VHDL and optimized in three stages. Measured against an unoptimized Wallace baseline and an inferred $A * B$ reference synthesised under identical conditions, the final architecture reaches a fastest derived zero-slack period of 1.75 ns against 1.63 ns for the reference. Plain Dadda is the fastest structural variant at 1.73 ns. The final architecture’s smallest area is $5029 \mu\text{m}^2$ against $4441 \mu\text{m}^2$, recovering 69.1% of the baseline-to-reference area difference.

The three contributions are unequal, and their sizes do not follow the size of the structural change each one makes. Sign-extension elimination removes 29.9% of the partial-product bits but only 5.8% of the area, because constant propagation removes most of the associated logic independently of it. The Dadda schedule contributes a comparable area reduction and the largest single improvement in the derived period. Fusing the Booth selector contributes the largest area reduction. Its mapped netlist contains 1145 fewer XOR/XNOR cells and 331 more FA_X1 cells than the separate-negation Dadda variant at the 3.0 ns constraint. At the same target, dynamic power falls from 8.25 mW to 4.61 mW. These paired changes support the conclusion that the flatter RTL description improved adder recognition.

The broader lesson is that RTL boundaries influence what the synthesis tool can recognise. The source-level cost of conditional inversion cannot be estimated reliably by counting XOR operators alone because the surrounding cone may map differently after the rewrite.

The next step is physical design. Placement, clock-tree synthesis, routing and extracted-parasitic analysis will provide more precise final area, timing and power values.

A Project structure and variant selection

Path	Purpose
rtl/	structural multiplier, reduction trees and final adder
cfg/	matching design, testbench and synthesis configurations
tb/ and sim/	self-checking RTL simulation
synthesis/syn/reports/	canonical timing, area and cell reports
synthesis/syn/reports_power/	activity-based power results
synthesis/plot_pareto.py	report parsing and Pareto figures

NBIT in `common_pkg` is the single width parameter. The RTL, reduction schedule and testbench derive their dimensions from it. Each design configuration has matching simulation and synthesis configurations, which keeps the architecture verified in simulation identical to the one synthesised.

Variant	Selected implementation
Wallace baseline	sign-extended rows and Wallace reduction
Wallace optimized	compact rows and Wallace reduction
Dadda	compact rows and Dadda reduction
Dadda fused selector	fused row selection and Dadda reduction
Behavioural Booth	Booth rows accumulated as an adder chain
Inferred reference	registered signed $A * B$

B Measurement provenance

Timing and area use the canonical synthesis report tree; power uses the corresponding power report tree:

```

synthesis/syn/reports/
synthesis/syn/reports_power/

```

Timing files follow `<type>_<configuration>_<period>.rpt`, with p replacing the decimal point in the period.

Tag	Source	Variant or measurement set
[R1]	synthesis reports	Wallace baseline
[R2]	synthesis reports	Wallace with sign-extension elimination
[R3]	synthesis reports	Dadda with separate negation
[R4]	synthesis reports	Dadda with fused selector
[R5]	synthesis reports	behavioural Booth adder chain
[R6]	synthesis reports	inferred signed multiplier
[R7]	3.0 ns reports	mapped-cell census for all six variants
[R8]	power reports	SDF/VCD/SAIF activity-based power results

Quantity	Source file	Value used
Area	<code>area_*.rpt</code>	total cell area
Cell counts	<code>reference_*.rpt</code>	count by cell family
Achieved period	<code>timing_*.rpt</code>	requested period minus slack
Dynamic power	<code>results_*.csv</code>	internal plus switching power
Leakage power	<code>results_*.csv</code>	leakage power